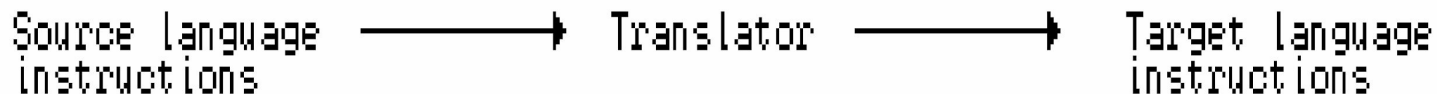


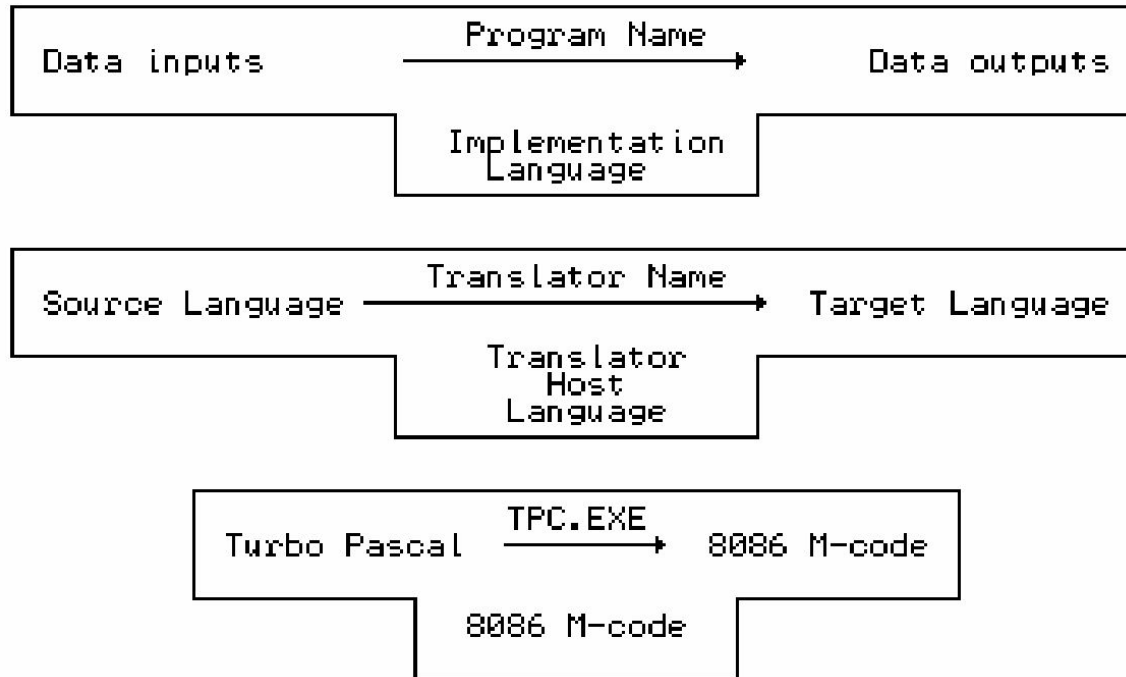
Introduction

- At least three Languages are involved in the development of translators: the source language to be translated, the object or target language to be generated, and the host language to be used for implementing the translator.
- If the translation takes place in several stages, there may even be other, intermediate, languages.



T-diagrams

- A useful notation for describing a computer program, particularly a translator called as T-diagrams.



M-code:
machine
code

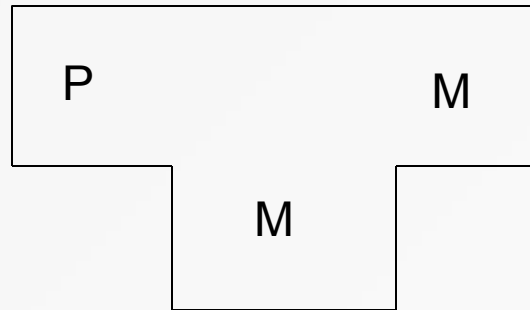
1. A General Program
2. A General Translator
3. A Turbo Pascal Compiler For an MS-DOS System

Bootstrapping

- This process, by which a simple language is used to translate a more complicated program, which in turn, may handle an even more complicated program and so on, is known as bootstrapping.

Bootstrapping

- Example: We wish to have a Pascal compiler for a new Processor, M.

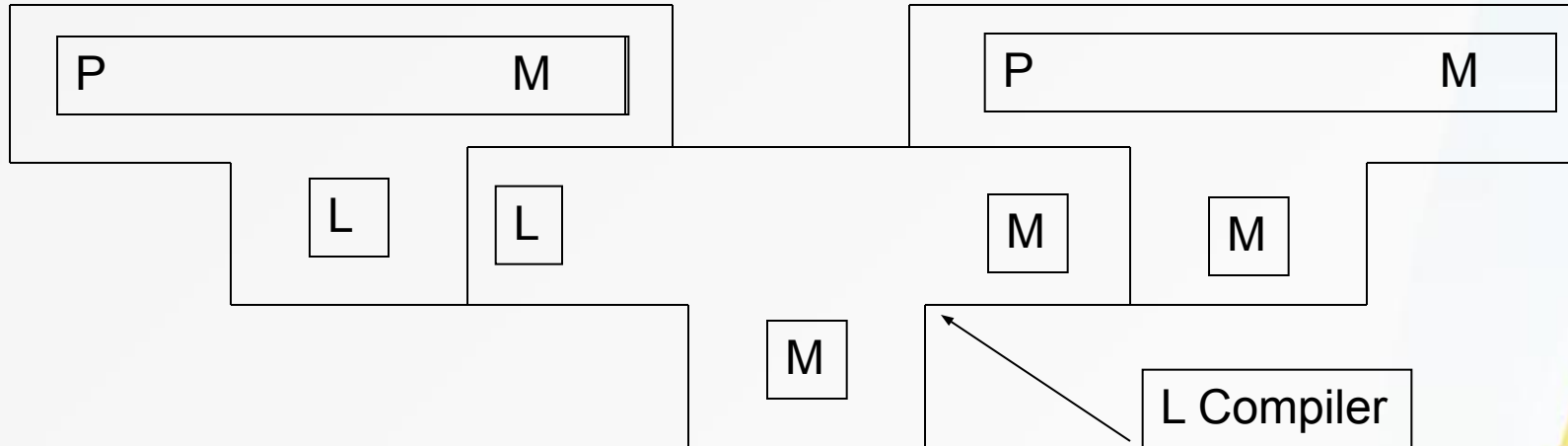


How do we get it?

Possibility #1: Write the compiler directly by hand, in machine language. In assembly language?

Bootstrapping

Possibility # 2: Write the Pascal compiler in some other language L, that is already implemented on M. Then, compile the compiler.

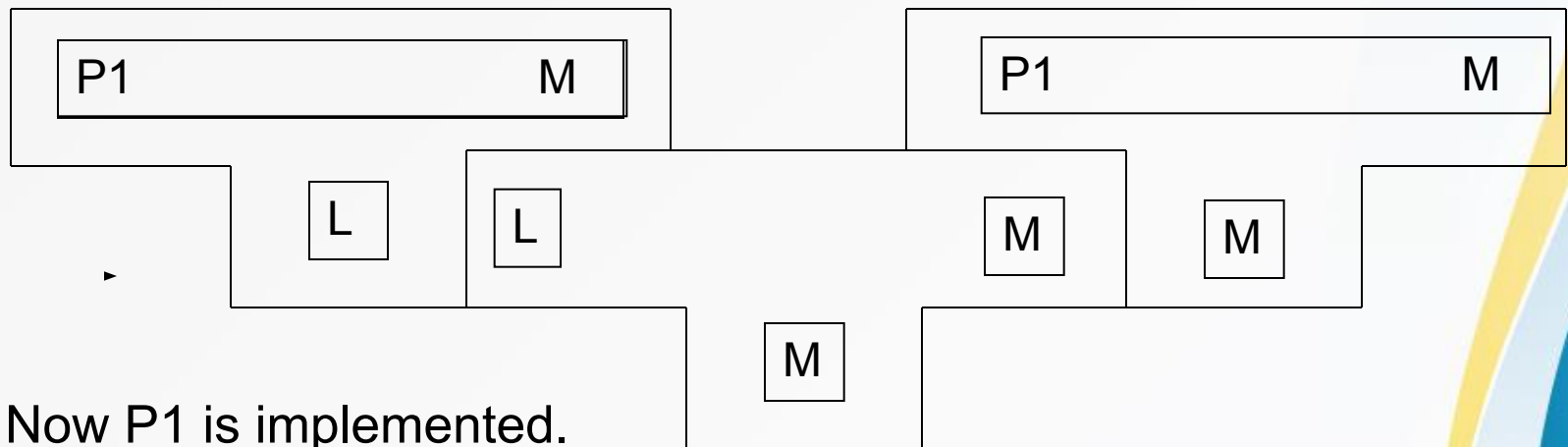


- But, ... this is probably too much work.

Bootstrapping

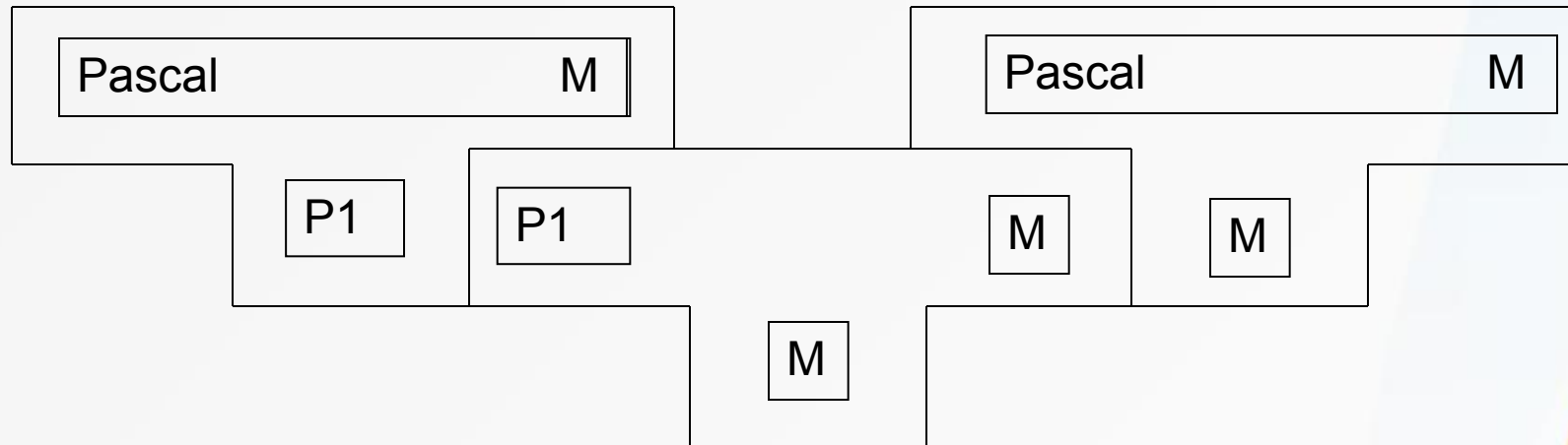
Possibility # 3: First, implement P1, a subset of Pascal. Then, write the desired compiler in P, and compile it.

Step 1:



Bootstrapping

Step 2:



- If P1 is chosen well, the P1 $\rightarrow$ M compiler and the Pascal $\rightarrow$ M compiler (in P1) together, are easier to write than the Pascal $\rightarrow$ M (in L) compiler.
- Theoretically, this process can go on indefinitely, bootstrapping up the level of the language.

Full Bootstrapping

- A full bootstrap is used when we have to build a new compiler from scratch.

□ **Example:**

- We want to implement an Ada compiler for machine M .
- We don't currently have access to any Ada compiler (not on M , nor on any other machine).

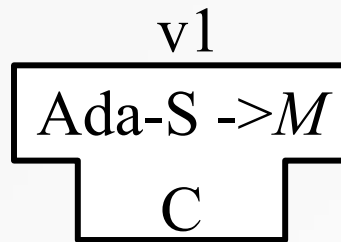
□ **Idea:**

- Ada is very large, let us implement the compiler in a subset of Ada
- And bootstrap it from a subset of Ada compiler in another language. (e.g. C)

Full Bootstrapping Cont..

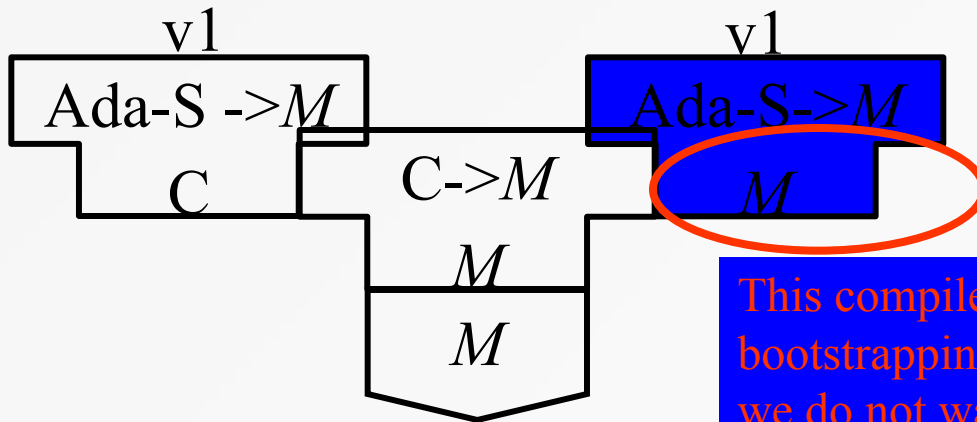
- **Step 1:** build a compiler for Ada-S in another language

Step 1a: build a compiler (v1) for Ada-S in another language.



Full Bootstrapping Cont..

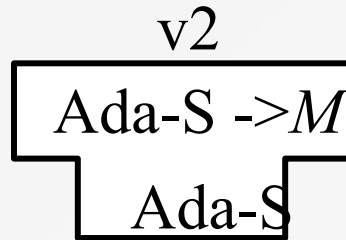
Step 1b: Compile v1 compiler on M



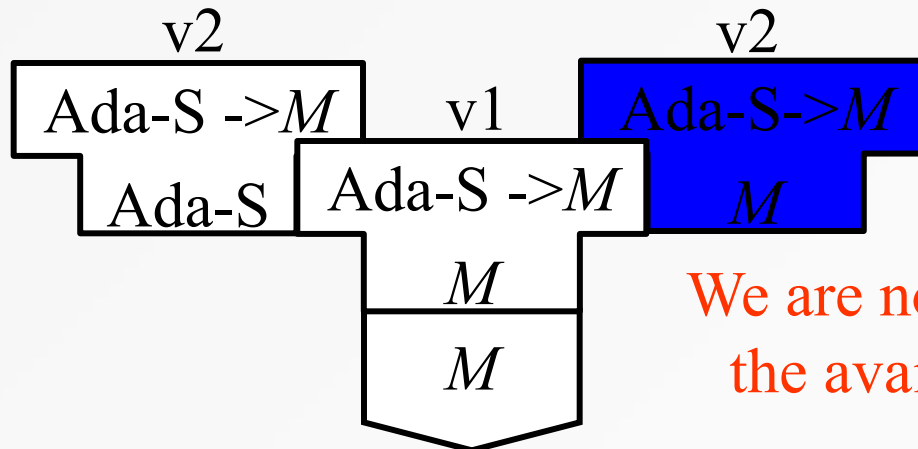
This compiler can be used for bootstrapping on machine M but we do not want to rely on it permanently!

Full Bootstrapping Cont..

Step 2a: Implement v2 of Ada-S compiler in Ada-S



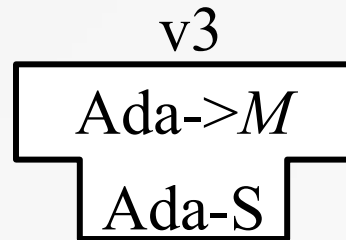
Step 2b: Compile v2 compiler with v1 compiler



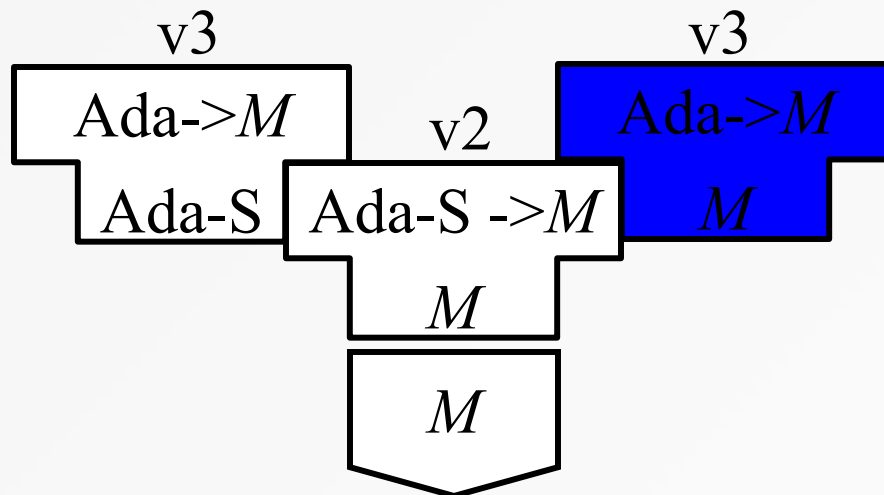
We are now no longer dependent on the availability of a C compiler!

Full Bootstrapping Cont..

Step 3a: Build a full Ada compiler in Ada-S



Step 3b: Compile with v2 compiler



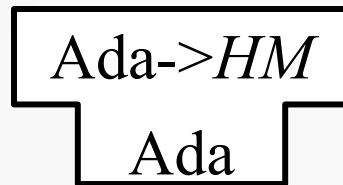
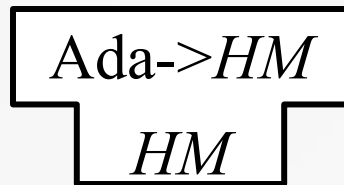
From this point on we can maintain the compiler in Ada.
Subsequent versions v4,v5,... of the compiler in Ada and compile each with the previous version.

Half Bootstrapping

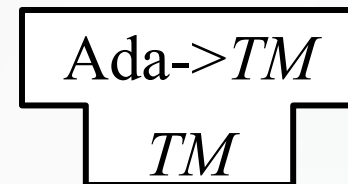
We discussed full bootstrapping which is required when we have no access to a compiler for our language at all.

Q: What if we have access to an compiler for our language on a different machine HM but want to develop one for TM ?

We have



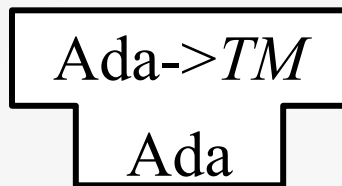
We want



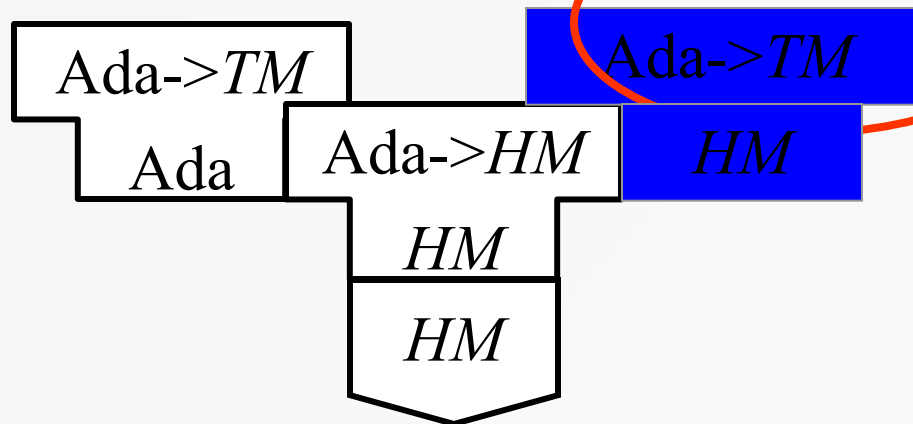
Idea: we can use cross compilation from HM to TM to bootstrap the TM compiler

Half Bootstrapping Cont..

Step 1: Implement Ada->TM compiler in Ada



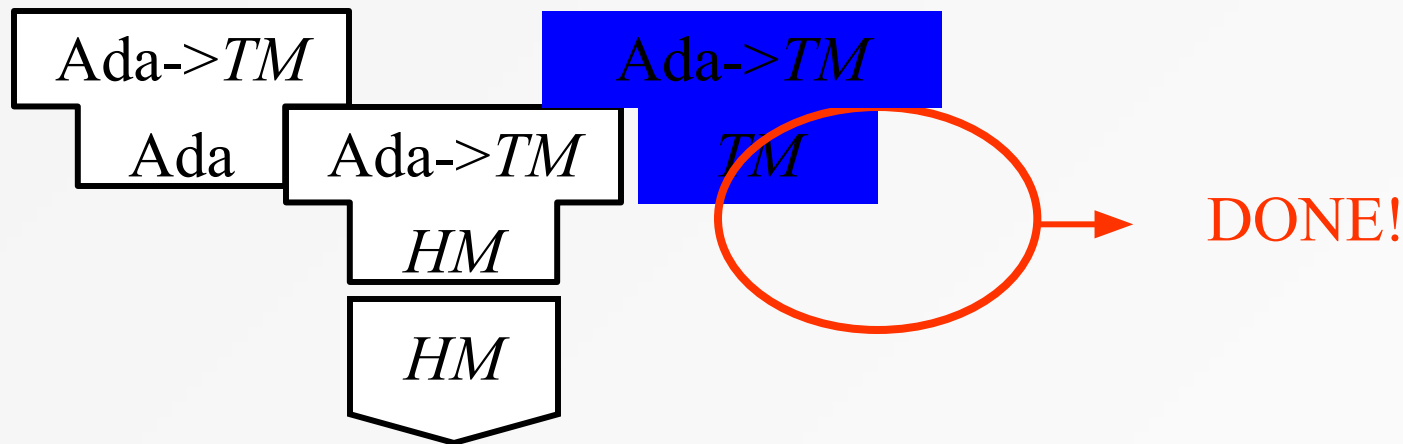
Step 2: Compile on *HM*



Cross compiler:
running on *HM* but
emits *TM* code

Half Bootstrapping Cont..

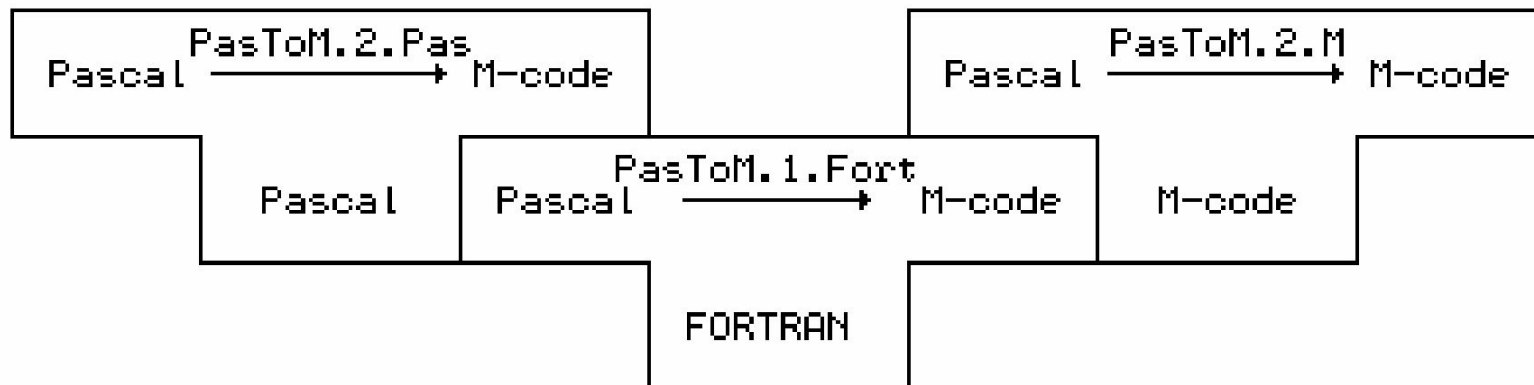
Step 3: Cross compile our *TM* compiler.



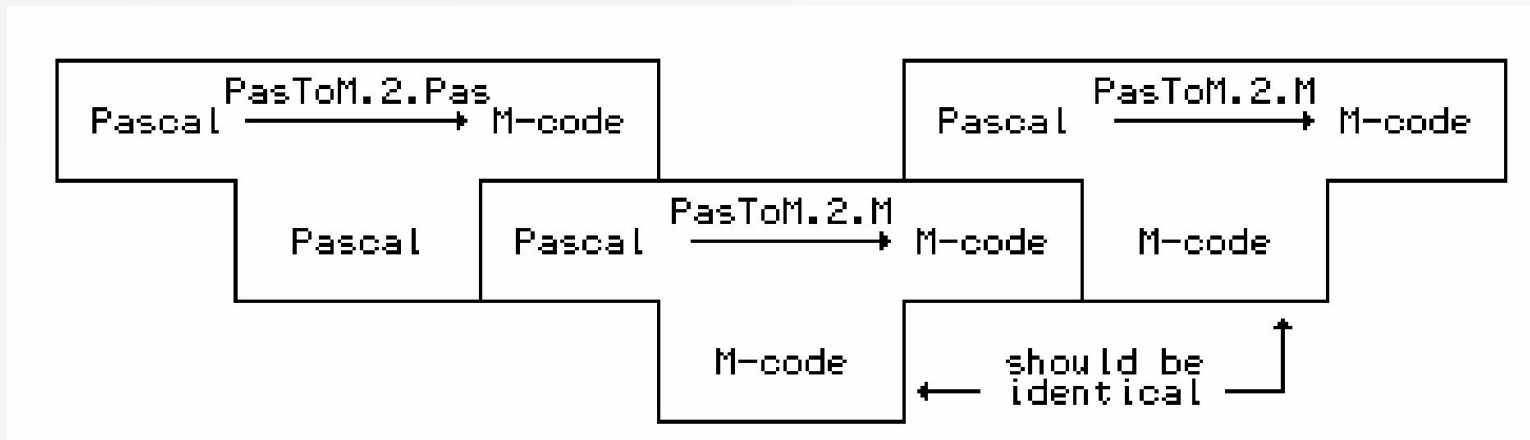
From now on we can develop subsequent versions of the compiler completely on *TM*

Self-compiling Compilers

- Once we have working system, we can begin using it to improve itself.
- Many compilers for popular languages were first written in another implementation language and then they rewritten in their own source language.



Self-compiling Compilers



- Once the development is done, it provides a fairly exhaustive self-consistency check, for if the compiler is used to compile its own source code, it should, of course, be able to reproduce its own object code.

Bootstrap to Improve Efficiency

The efficiency of programs and compilers:

Efficiency of programs:

- memory usage
- runtime

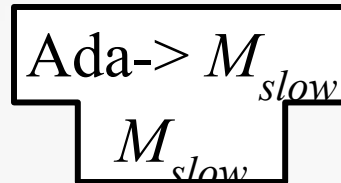
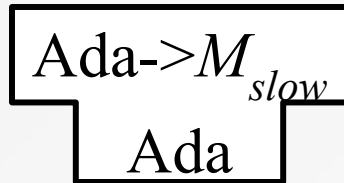
Efficiency of compilers:

- Efficiency of the compiler itself
- Efficiency of the emitted code

We start from a simple compiler (generating inefficient code) and develop more sophisticated version of it. We can then use bootstrapping to improve performance of the compiler.

Bootstrap to Improve Efficiency

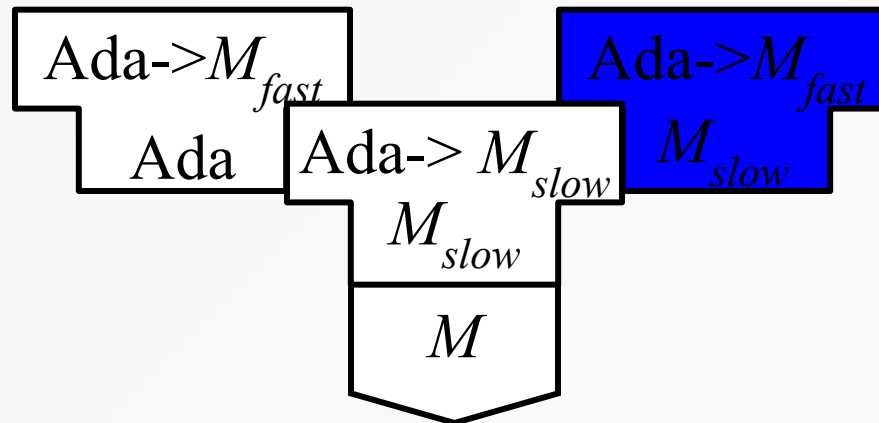
We have



We want

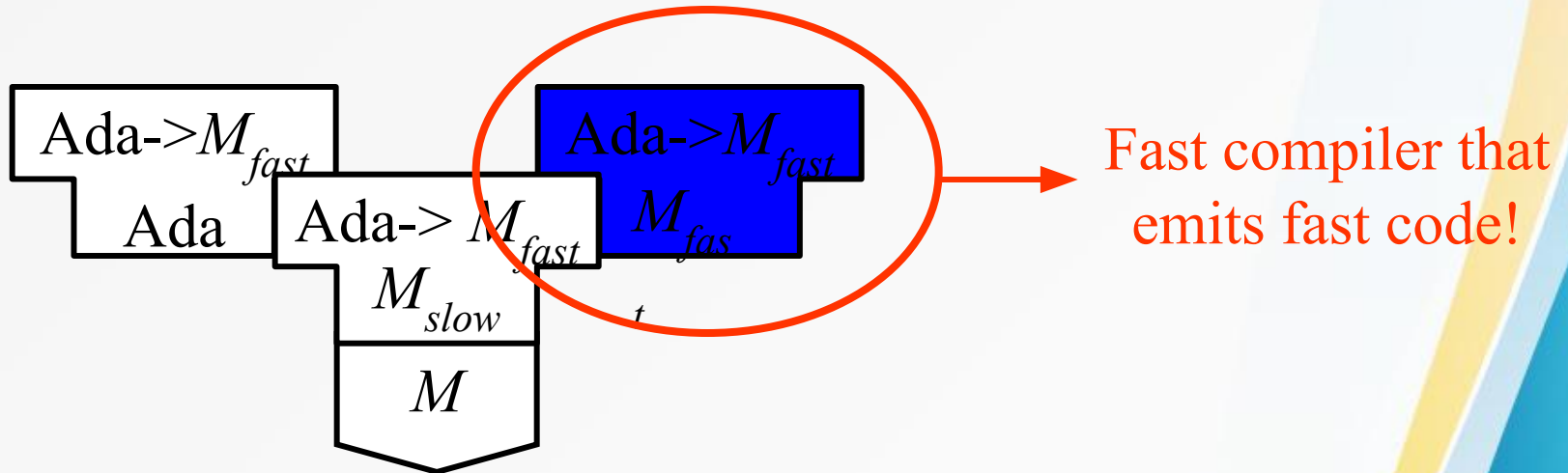


Step 1



Bootstrap to Improve Efficiency

Step 2



Conclusion

- To write a good compiler you may be writing several simpler ones first
- You have to think about the source language, the target language and the implementation language.
- The work of a compiler writer is never finished, there is always version 1.x and version 2.0 and ...